Scheinaufgabe 1 (33%): Stack, Heap, Global (Thema: „storage duration“)?
Welche Zahlen (gelb hervorgehoben) liegen in welchem Speicher (ein “x” pro Spalte)?

	1	2	3	4	5	6
Stack			x	x		
Heap		x			x	x
Global	x					

```
#include <iostream>
#include <memory>

int a{1};
int *b=nullptr;

void f() {
     b = new int{2};
     int c{3};

     std::cout << a << "\n";
     std::cout << *b << "\n";
     std::cout << c << "\n";
}

int main() {
     f();
     int d{4};
     std::shared_ptr<int> e = std::make_shared<int>(5);
     std::unique_ptr<int> f = std::make_unique<int>(6);
     std::cout << d << "\n";
     std::cout << *e << "\n";
     std::cout << *f << "\n";
     delete b;
}
```

Scheinaufgabe 2 (33%): Was läuft hier falsch (Thema: „storage duration“)?

```
#include <iostream>
int *cnt;
void f() { int i{99}; cnt = &i; } // On my computer:
int main() {                      // (may differ on your computer)
     f();
     std::cout << *cnt << "\n";   // → 99
     std::cout << *cnt << "\n";   // → 1 ?????!?!?!?
}                                      'bei mir lebt 'O'
```

- i = lokale Variable → liegt auf Stack, mit Ende von f() wird ihr Speicher wieder freigegeben
- cnt zeigt aber immer noch dorthin = dangling Pointer
- Beim ersten Aufruf steht zufällig an der Stelle noch 99. Jedoch benutzt 'cout' die Stelle selbst und überschreibt den alten Platz
  → deshalb steht beim 2. Aufruf 'O'

Scheinaufgabe 3 (33%): Klausuraufgabe (Thema: „storage duration“)

```
#include <iostream>
#include <memory>

struct Point {
  float x,y;
  Point(float _x, float _y) {
    x=_x; y=_y;
  }
  ~Point() {
    std::cout << "destroy " << x << "," << y << "\n";
  }
};

struct A {
  Point *p;
};
struct B {
  std::shared_ptr<Point> p;
};
struct C {
  std::unique_ptr<Point> p;
};

int main() {
  A a1,a2;    B b1,b2;    C c1,c2;

  a1.p = new Point(11,11);                 //                 11
  a2.p = new Point(12,12);                 //              12  |
  b1.p = std::make_shared<Point>(21,21); //           21  |   |
  b2.p = std::make_shared<Point>(22,22); //        22  |   |   |
  c1.p = std::make_unique<Point>(31,31); //     31  |   |   |   |
  c2.p = std::make_unique<Point>(32,32); // 32  |   |   |   |   |
                                           //  |   |   |   |   |   |
  a1.p = a2.p;                             // [ ][ ][ ][ ][ ][ ]
  b1.p = b2.p;                             // [ ][ ][ ][X][ ][ ]
  // c1.p = c2.p;  // kompiliert nicht!
}                                          // [X][X][X][ ][ ][ ]
```

nie - Memory Leak - kein Defekte Pointer

Es werden sechs Punkte („Point“-Objekte) auf dem Heap erzeugt. Wann werden diese freigegeben? Kreuzen Sie das Kästchen in der Zeile an, in der der entsprechende Punkt (Koordinate 11,12,21,22,31,32) freigegeben wird.

Warum kompiliert die letzte (auskommentierte) Zeile nicht (siehe Code)?

unique_ptr = alleiniger Besitzer des Objekts. Daher funktioniert der Kopierkonstruktor nicht

_____ relevante Foliennummer aus 03_slides_classes_and_objects1.pdf: _____